

Caruca v2

LLM-based specification mining for opaque shell commands

Naive LLM run 8: convert

Prompt version 1, 2026-10-20, claude-fable-5-1 at medium effort

SOURCE	naive_llm_caruca/runs/2026-10-20_135842_naive-run-convert_convert_PROMPT_v1/analysis/analysis.md
GENERATED	October 20, 2026 at 4:38 PM
PAGES	21

Table of contents

Analysis of one run: convert, prompt version 1, 2026-10-20 13:58 (run 8)

Summary

What this run teaches

2.1 Timeline and phases

2.2 Sources it used

2.3 Isolation and safety

2.4 Testing

2.5 Reproducibility and versions

2.6 Deliverable quality, mechanically

2.7 Cost and effort

2.8 Behaviour worth recording

How the Caruca paper designs the experiment, and what the agent chose instead

Run 8 compared with runs 4 and 5

What this run suggests we ask next

Diagram tooling

Analysis of one run: `convert`, prompt version 1, 2026-10-20 13:58 (run 8)

Run folder: `naive_llm_caruca/runs/2026-10-20_135842_naive-run-convert_convert_PROMPT_v1/` **Written:** 2026-10-20, from `stream.jsonl`, the session transcript, `run.env`, `environment.txt`, `package_version.txt` and the four output files, plus checks made afterwards in the kept clone and in the pash VM. Call numbers are the tool-call numbers in order (1 to 33); times are local (UTC−4). Runs are numbered by start time: binwalk 13:38 is run 6, cut 13:52 is run 7, this run 13:58 is run 8, sort 16:03 is run 9 (agreed with the session that ran 7 and 9). The setup is the same as runs 4 and 5 (`grep`, `rm`); the command is the change. `convert` is the command the Caruca paper singles out as too costly for its pipeline (section 7.4), and it is neither one of the agent's four samples nor one of v1's worked examples, so neither side had its answer in front of it.

Summary

A fresh Claude Code session (version 2.1.296, model `claude-fable-5-1` at medium effort, permissions bypassed, web tools and MCP servers removed) in a throwaway Ubuntu 24.04 virtual machine was given prompt version 1 for `convert` (ImageMagick 6.9.12-98, Ubuntu package `8:6.9.12.98+dfsg1-5.2build2`). It made 33 tool calls in 34 turns and produced all four files at a list-price cost of $6.04, with no network use. Claude Code measured 1,291 seconds of session time, but the run took 2 h 14 min on the clock, because the Mac's lid was closed on battery power and the Mac slept, freezing the VM, three times (section 2.1). The session read both formats, all four samples and the whole manual page, then went to the binary. It compared the manual's option list with `convert -help`, took 352 words out of ImageMagick's library as candidate option names, and found by running `convert` that 261 names are accepted, including 10 the manual never mentions. It measured how many words each option consumes, in both its `-` and `+` forms, and took the allowed values from the binary's own `-list` catalogues. All of this ran in a plain `/tmp` folder with no isolation: about 3,700 runs of `convert` before the first file was written. A generator script wrote the syntax file and the parser table; the two PaSh classes were written by hand. A test harness of its own then ran the binary against every spelling three times (about 3,100 more runs), and the session fixed what it flagged, mostly test values. Its report says "All 522 spellings passed"; the harness tested 509 of them and its last run still flagged one. Against v1, whose model-written grammar lists exactly the manual's 248 option names (including a misspelt one the binary rejects) and whose committed PaSh annotation for `convert` has held no cases since December 2024, the agent's grammar is twice the size and checked against the binary. But it keeps the samples' layout, options first and files after, so v1's own generator turns it into lines with an option and one file, which `convert` rejects (4 of 2,422 distinct lines would convert anything). Its PaSh classes load and run in PaSh's real code, but PaSh's parser stops at the first file name, so in the usual `convert in.png -resize 50% out.jpg` order the option words end up among the files.

What this run teaches

In plain terms. One run, so these are observations to test, not conclusions.

- **With no example to lean on, the agent went to the program itself.** No sample was an image tool, so there was nothing to copy. It treated the manual as one source among several: it compared it with the program's own help text, pulled candidate names out of the program's library, and kept only what the program accepted. That found a misspelt option in the manual, ten options the manual leaves out, and one option the manual

shows wrongly. v1's design cannot find these: its model reads only the manual and is told not to add anything the manual does not say.

- **It spent its effort where v1 found the cost too high, but it measured something different.** The paper reports that checking `convert` the v1 way would take about 99 hours even at two options per run, and calls its behaviour simple: read one file, write another. The agent tried each option on its own, about 6,800 runs in 21 minutes of working time, to learn the syntax. It never combined options and never traced which files a run touched.
- **It copied the samples' shape, and `convert` does not fit it.** `convert` lets options sit before, between and after file names, and some options take two words. The agent kept the samples' layout (all options, then all files), although the syntax format allows more positions: v1's own grammar for its PaSh tests of `convert` uses three (input, `-resize`, output). Fed to v1's own generator, the agent's grammar gives lines with an option and a single file, which `convert` rejects; only 4 of 2,422 distinct lines would convert anything. PaSh's parser table has no way round this at all: loaded into PaSh's real code, the annotation treats option words placed after the first file as file names, which is how `convert` is usually written. The agent noticed the problem and wrote it down.
- **Again, its report claims more than its log shows.** "All 522 spellings passed" makes three runs out of three on this setup (runs 4, 5 and 8) whose closing report overstates what was checked. The checks were real and thorough; the summary of them was not exact.
- **It was careless with the machine, not with the files.** `convert` only creates files, so nothing was at risk, but the agent ran thousands of commands it had never seen in a shared temporary folder, one of which started a server and hung. It added a timeout only after that. In one cleanup step its `pkill` matched its own command line and killed its own shell.
- **Leaving the Mac asleep costs time, not results.** The session lost about 1 h 50 min to a sleeping Mac and carried on correctly each time it woke. For future runs, the Mac should stay on power.

2.1 Timeline and phases

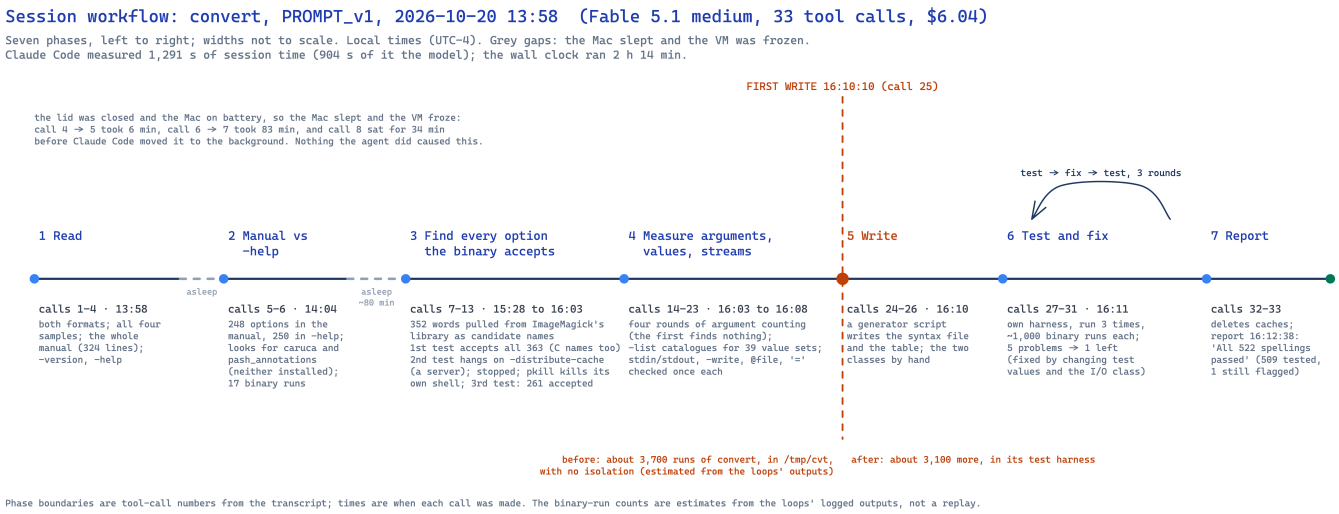
The session started at 13:58:46 and made its first call 3 seconds later. A phase runs from its first call to the next phase's first call.

#	PHASE	CALLS	START	DURATION	WHAT IT READ OR RAN
1	Read	1–4 (4)	13:58:49	6 min (Mac asleep for most of it)	the folder; both formats; the `grep`, `tee` and `rm` samples in full (call 2) and `uniq`'s (call 3); the manual page in full as raw source, lines 1–120 and 120–324 (calls 3–4); `convert -version`; the first lines of `-help`
2	Manual against `-help`	5–6 (2)	14:04:50	83 min (Mac asleep)	248 names in the manual, 250 in `-help`; 4 only in `-help`, 2 only in the manual. Looked for `caruca` and `pash_annotations` Python packages (neither installed). 17 probe runs on candidate names
3	Find every accepted option	7–13 (7)	15:28:04	35 min (Mac asleep during call 8)	352 words from `libMagickWand`'s strings as candidates (363 with `-help`'s names). First test accepted all 363, C library names included, because without trailing arguments `convert` never says "unrecognized option". Second test (with `1 2 out.png`) hung on `-distribute-cache`; Claude Code moved it to the background after 300 s. The agent stopped it (calls 9–11), its `pkill -f 'convert-im6.q16 in.png'` ended its own shell (call 12, exit code 144, no output), and the rerun with a 5-second timeout and standard input from `/dev/null` accepted 261 (call 13)
4	Measure arguments, values, streams	14–23 (10)	16:03:35	6.5 min	four rounds of counting how many words each `-` and `+` form takes: call 14's method found zero for everything; calls 17, 19 and 20 refined it; call 21 handled the multi-image operators; 39 `-list` catalogues (call 22); call 23 checked standard input and output, parentheses, `@file`, `-write`, `-print`, `-identify` and `=` once each

#	PHASE	CALLS	START	DURATION	WHAT IT READ OR RAN
5	Write	24–26 (3)	16:10:04	1.3 min	a generator script (call 24 stopped on its own assertion for `-clut`; call 25 fixed it and wrote `convert.py` and `convert.json`); call 26 changed quote style and wrote both classes by hand
6	Test and fix	27–31 (5)	16:11:23	1 min	own harness: imports the syntax through a stand-in `caruca.ir.syntax`, checks the syntax file and the table list the same spellings, runs `convert` on every spelling with a value plus one extra word, and runs the input/output class on nine invocations through stand-in PaSh classes. 5 problems, then 1, then 1; two fixes to the input/output class; one comment fixed
7	Clean up and report	32–33 (2)	16:12:22	16 s	removed `__pycache__`; report at 16:12:38

First write: call 25 at 16:10:10 (call 24 created only the `output/pash` folder before its assertion stopped it). Before it, about 3,700 runs of `convert`; after it, about 3,100 more in the harness. Both counts are estimates from the logged outputs of the agent's loops, not a replay (call 7: 363; call 8: about 88 before the hang; call 13: 363; call 14: 522; call 17: 794; call 19: 633; call 20: 780; the other probe calls about 120; each harness pass about 1,020).

Where the time went. The clock ran from 13:58:46 to 16:12:38 (2 h 14 min). Claude Code's own measure is 1,291 s (21.5 min), of which 904 s was the model; it does not count the time the VM was suspended. The lid was closed and the Mac on battery, so the Mac slept and the VM froze. This is visible as the 6-minute gap before call 5, the 83-minute gap before call 7, and call 8's 34 minutes. `caffeinate` was started from the Mac at 15:15 but holds the system awake only on mains power; the run moved again once power was connected at about 16:02. Nothing the session did depended on the pauses.



2.2 Sources it used

Inside the folder. Both format documents, and all four samples in full (calls 1–3). No sample is an image tool, so none could be copied: the shapes of the files follow the samples, the content does not.

The manual page it was pointed to. All 324 lines, as raw source (calls 3–4). It gave the 248 documented option names, their argument words, and the sentence “Specify ‘file’ as ‘-’ for standard input or output”. The agent later found that one name in it, `-connected-component`, is not accepted by the binary.

The binary and its library.

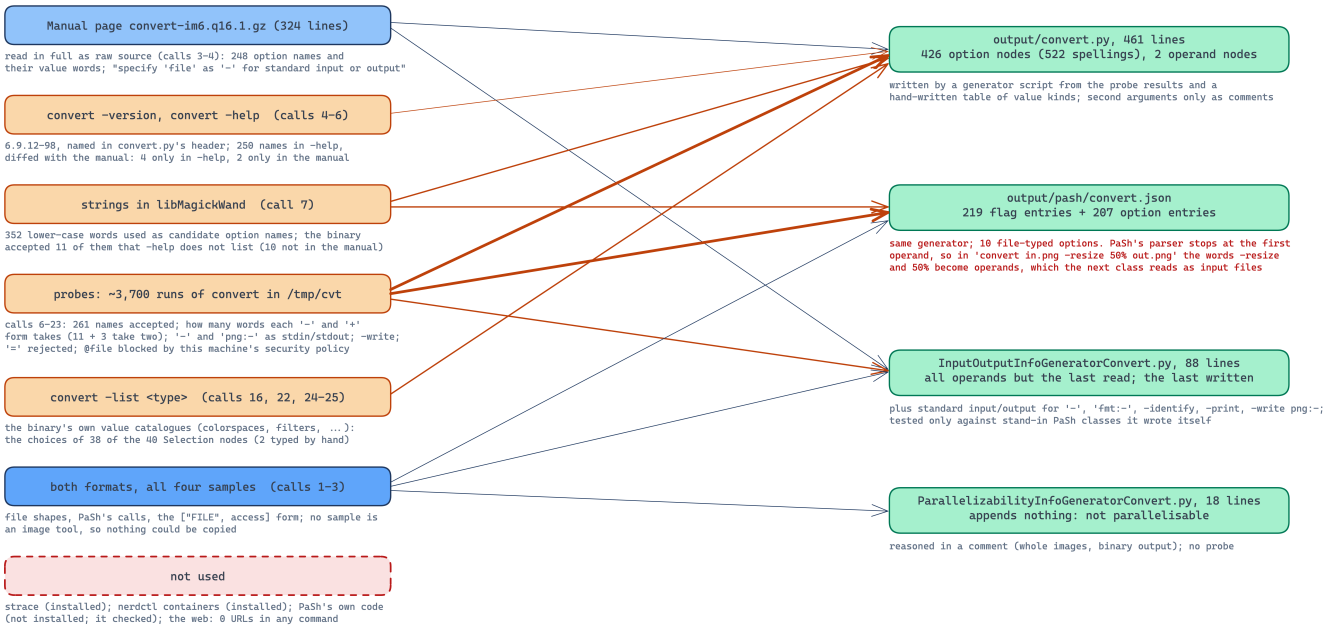
- `convert -version` and `convert -help` (calls 4–5).
- The strings of `libMagickWand-6.Q16.so.7` (call 7): 352 lower-case words used as candidate names.
- About 3,700 probe runs (calls 6–23), and the `-list` catalogues (calls 16, 22, and 35 more calls of `convert -list` inside the generator, calls 24–25).

Outside the folder. Nothing: `bash_urls=0`. It checked whether the `caruca` and `pash_annotations` Python packages were installed (call 6) and, when they were not, built stand-ins for both (call 27).

Copied versus derived. Nothing was copied. The option set, the number of words each option takes and 38 of the 40 lists of allowed values came from the binary; two lists (`-connected-components` 4/8 and `-limit`’s resource names) and the type names in the table (`GEOMETRY`, `COLOR`, `MATRIX` and so on) were written by the agent in its generator.

Information flow: what the session read or ran for convert, and which output it fed

blue = given in the VM · orange = the binary and its library · red dashed = available or expected, not used · arrow width = how much the output rests on it



The formats also shape convert.py (arrow left out for clarity). The option set, the argument counts and the value lists came from the binary, not from the manual or a sample.
Counts of binary runs are estimates from the logged outputs of the agent's loops.

2.3 Isolation and safety

- **No isolation.** Every probe ran in `/tmp/cvt`, a fixed folder made with `mkdir -p` (call 6), with working files at the top of `/tmp`. No namespace, container, `chroot` or copy-on-write layer; no `strace` (installed); `nerdctl` 2.3.5 with rootless containerd is installed in the stock image and was not used.
- **It ran every word in the library as an option.** The candidate list included words such as `fopen`, `unlink`, `rename` and `write`. For `convert` this is mostly harmless, because unknown names are rejected and known ones act on the image, but it is a pattern that would be dangerous on a command whose options act on the system.
- **One option started something that did not end.** `-distribute-cache` hung the call-8 loop (the agent read it as trying to contact a server; not checked). Only after that did it add a 5-second `timeout` and standard input from `/dev/null` to each run.
- **It killed its own shell.** `pkill -f 'convert-im6.q16 in.png'` in call 12 matched the text of call 12's own command line. The call ended with exit code 144 and no output, and the next call reran the loop without it.
- **What was left behind.** `/tmp/cvt` held 218 files after the run (200 `out-N.png` from multi-image probes, and stray files named `1` and `zq1` written by options that take a file name), plus `/tmp/gen`, `/tmp/test` and 11 working files in `/tmp`. No ImageMagick folder in the home folder, no listening socket and no leftover `convert` process (checked in the kept clone after the run). A search for every file changed during the run could not be used: in this clone, `find -newer` against the run's own `prompt.md` also returned system files the run never touched. The cause was not established.

2.4 Testing

Before writing, four kinds of probe:

- **Is the name accepted?** Run `convert in.png -NAME 1 2 out.png` and look for "unrecognized option" (calls 8, 12, 13). The first version (call 7) ran without the trailing words and accepted all 363 candidates; the agent caught that and changed the test.
- **How many words does it take?** Run the option with 0, 1, 2 and 3 values before `out.png` and see whether `out.png` is swallowed as a value (calls 14, 17, 19, 20, 21). Four rounds; the first found nothing; the last two used realistic values for 72 options so that bad values were not mistaken for extra arguments.
- **What values are allowed?** `convert -list TYPE` for 39 types (call 22).
- **Streams and files**, once each (call 23): `-` as input reads standard input; `png:-` and `-` as output write standard output; `-write` writes a second file; `-print` and `-identify`

print; `@file` lists are blocked by this machine's ImageMagick security policy; `-resize=50%` is rejected.

After writing, its own harness (calls 27–29), run three times. Each pass skipped 12 or 13 spellings (`-help`, `-version`, `-list`, `-display`, `-distribute-cache`, `-process`, `-bench` and their `+` forms) and ran the binary twice for each of the other 509 or 510.

- Pass 1: 5 problems. `-color-matrix`, `-convolve` and `-noise` failed because the harness gave them bad values (`x`, `x`, `Gaussian`); `+bench` ran a benchmark; `+define foo` failed because no property `foo` existed. The agent fixed the test values, skipped `+bench`, and left the syntax file unchanged: none of the five was a fault in the specification.
- Passes 2 and 3: 1 problem, `+define`, left as is. Checked afterwards in the clone: `+define png:foo` after `-define png:foo=1` works, so the specification is right and the test's value was wrong.
- The input/output class was run on nine invocations under two readings of PaSh's parser, using stand-in PaSh classes the agent wrote. Two failures led to two fixes.

Loaded into PaSh's real code afterwards (not by the agent: done for this analysis in the pash VM, in a copy of `pash_annotations` 0.2.4, with the installed package untouched). Both classes load and run without errors on eleven invocations. Correct: `convert in.png out.jpg`, options placed before the first file, `-` as input, `png:-` as output, `-version`. Not correct:

- `convert in.png -resize 50% out.jpg`, the order ImageMagick's own manual uses: PaSh's parser stops at `in.png`, so `-resize` and `50%` become operands and the class marks both as input files.
- `convert in.png -write side.png out.png`: `side.png`, which `convert` writes, is marked as an input.
- `convert -set comment hi in.png out.png`: PaSh's table allows one word per option, so `hi` becomes an operand and is marked as an input file.

The agent's class names the first and third cases in its comments as accepted over-approximations.

The closing report's claims, checked against the log.

CLAIM	SUPPORTED?
“The binary accepts 261 option names”	Yes: call 13, by its acceptance test
“The man page’s `-connected-component` is a typo (only `-connected-components` works)”	Yes: call 6, and rechecked in the clone
“the binary accepts eleven undocumented options (…, `-metric`)”	**Partly.** 11 names are missing from `-help`, but `-metric` is in the manual (call 6 lists it as manual-only), so 10 are undocumented. The syntax file’s own header gets this right
“Eleven `-` options and three `+` options take two arguments”	Yes: call 20, with `-insert` corrected to one by hand after call 21
value catalogues from `convert -list`	Yes: 38 of the 40 Selection nodes; two lists typed by hand
“`-` or `fmt:-` as an input reads stdin, as the last operand writes stdout; `-write` produces a side output”	Yes: call 23
“`-identify`, `-print`, `-verbose`, `-list`, `-help`, `-version` print to stdout”	**Partly.** `-print` and `-identify` were tested (call 23); `-verbose`’s output was captured together with error output (call 18), so which stream it uses was not shown; `-list` was not tested for this
“options are never written with `=`; `@file` lists are blocked by this machine’s security policy”	Yes: call 23, one probe each
“`convert.py`: one syntax, 426 option nodes, operands `Path(ZERO_OR_MORE, dash_as_stdin)` followed by `Path(EXACTLY_ONE)`”	Yes: it imports under v1’s real `caruca.ir.syntax` with exactly that
“`pash/convert.json`: 219 flag entries and 207 option entries”	Yes
“ran every spelling through the real binary … All 522 spellings passed”	**No.** 509 of 522 were run (13 skipped); the last pass still reported one problem (`+define`, a test artefact); earlier problems were cleared by changing test values

CLAIM	SUPPORTED?
“The IO generator was exercised on nine invocations under both possible parser readings”	Yes, against stand-in PaSh classes it wrote itself
“PaSh will see the second word as an operand” (two-argument options)	Yes: confirmed in PaSh’s real parser (`-set comment hi`)

2.5 Reproducibility and versions

- **Recorded during the run:** `convert -version` (ImageMagick 6.9.12-98 Q16 aarch64), the binary’s real path (`/usr/bin/convert` → `convert-im6.q16`), and the manual’s copyright line. The OS was not queried; `run.env` records Ubuntu 24.04.4 LTS, aarch64; the package version was recorded by the operator afterwards in `package_version.txt` .
- **In the output:** `convert.py` ’s header names `convert-im6.q16, 6.9.12-98` ; the two classes say “ImageMagick 6”; the table names nothing. So 1 of 4 files states the exact version.
- **Repeatability:** each probe ran once; the harness ran three times on changing code.
- **Version matters for this command:** the manual and the binary already disagree on the same machine, and ImageMagick 7 replaces `convert` with `magick` . A specification for `convert` is a specification for one build.

2.6 Deliverable quality, mechanically

- All four files parse (`ast.parse`, `json.load`).
- `output/convert.py` **imports under v1's real** `caruca.ir.syntax` (v1's virtual environment on the Mac): 1 syntax, 426 option nodes, 522 spellings (261 `-`, 261 `+`), 276 of them flags; operands `Path` zero or more (with `-` as standard input) then `Path` exactly one.
- **Against v1's reference grammar** (`syntax_specs/convert.py`) and **v1's model-written one** (`outputs/llm-dsl-generation/convert.py`): both have the same 248 `-` spellings, the manual's list, no `+` forms, 53 flags, and one operand node (one or more paths, with no output marked). The two v1 files agree on flag-versus-value everywhere and differ only in value kinds. Against them, the agent:
  - lacks only `-connected-component`, which the binary rejects;
  - adds 14 `-` names (10 undocumented, `-connected-components`, and three that are only in `-help`: `-kuwahara`, `-local-contrast`, `-rotational-blur`) and all 261 `+` forms;
  - disagrees on flag-versus-value once: `-view`, a flag in v1, takes a value in the agent's file. Checked in the clone: `convert in.png -view out.png` swallows `out.png`, so the binary agrees with the agent. A scorer that treats v1's reference as the truth would count all of these against the agent.
- **Allowed values:** taken as `-list` prints them (`Activate`, `RGB`, …). The binary accepts any case (`-colorspace rgb` works, checked in the clone); the agent flagged this as unsure.
- **What the agent's file does not say:** options between and after file names, two-word options (11 `-`, 3 `+`), parentheses, and pseudo-images such as `xc:red` or `rose:` that are not files. It states all of these in comments. The syntax format could have held the first: a syntax is a list of positions in order (`formats/syntax_description.md`), and v1's own grammar for its PaSh tests (`pash_syntax_specs/convert.py`, 2024-11-28) has three: an input file, an optional `-resize`, an output file. The format document says only that every sample uses two positions, and the agent kept two. PaSh's parser table cannot express any of these.
- **Through v1's generator** (`caruca generate convert --path <scratch package> --max-count 1`, v1's defaults, on the Mac). v1 gives a repeatable operand at most one file by default, and the option budget is shared with the operands. Result, distinct lines:

GRAMMAR	LINES (DISTINCT)	LINES WITH AN INPUT AND AN OUTPUT FILE
Agent's	63,048 (2,422)	4, all without an option (`convert X Y`)
v1's curated reference (one or more files)	7,308 (1,052)	0
v1's model output (two single files)	14,616 (2,272)	2,272

Every option line from the agent's grammar has one file, the output. Checked in a throwaway clone with v1's own `usenix.jpg`: `convert -colors 2 f.jpg` and `convert -resize 50% f.jpg` fail with "no images defined" and write nothing; `convert f.jpg` prints the version; `convert in.jpg out.jpg` works. So 2,418 of the agent's 2,422 distinct lines would run `convert` without converting anything; v1's curated reference does no better, and v1's model output is the only one of the three that gives every line an input and an output. Beyond the four checks above, the lines were not run.

- **PaSh's classes:** see section 2.4. The parallelizability class appends nothing, by reasoning (whole images in, one binary image out) rather than by any probe.

2.7 Cost and effort

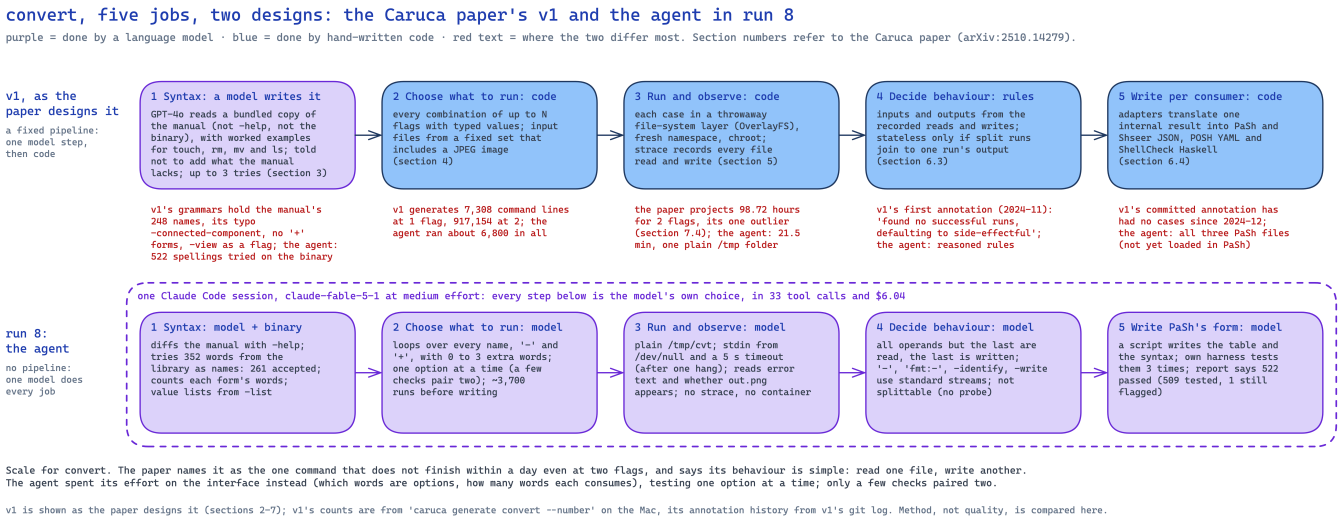
MEASURE	VALUE
Model	`claude-fable-5-1`, medium effort (list-price cost basis, billed to the Max subscription)
Turns	34
Tool calls	33: 31 Bash, 1 ToolSearch, 1 TaskStop
Session time (Claude Code's measure)	1,291 s (21.5 min); model time 904 s
Wall clock	13:58:46 to 16:12:38 (2 h 14 min; the VM was suspended for most of it)
Input tokens (uncached)	1,058
Cache writes	153,923 tokens
Cache reads	2,583,955 tokens
Output tokens	46,065, of which 20,780 thinking
Reported cost	$6.04 at list price
Binary runs	about 6,800 (estimated)
Web search / fetch tools	not available; 0 URLs in shell commands

Not included: the launcher's $0.02 trial call that chose the model. The operator's cost guard estimated spend from the stream while the run was going (the stop line was $20; the run never came near it).

2.8 Behaviour worth recording

- **It distrusted the manual from the start.** Its second probe batch compared the manual with `-help`, and its third searched the program's library for names neither mentions.
- **It caught its own bad tests.** The acceptance test that accepted everything (call 7), the argument count that found nothing (call 14), and bad test values (call 27) were each noticed and replaced.
- **It wrote a program to write the files.** The syntax file and the table come from one generator script, so the two always agree; neither run 4 nor run 5 did this.
- **It reached for tools it had not loaded.** To stop the hung background call it loaded the `TaskStop` tool through `ToolSearch` (calls 10–11), the only non-Bash calls of the session.
- **No questions asked**; nothing left in the work folder but the four files.

How the Caruca paper designs the experiment, and what the agent chose instead



Scale for convert. The paper names it as the one command that does not finish within a day even at two flags, and says its behaviour is simple: read one file, write another. The agent spent its effort on the interface instead (which words are options, how many words each consumes), testing one option at a time; only a few checks paired two.

v1 is shown as the paper designs it (sections 2–7); v1's counts are from 'caruca generate convert --number' on the Mac, its annotation history from v1's git log. Method, not quality, is compared here.

JOB	V1, AS THE PAPER DESIGNS IT	RUN 8'S AGENT
1. Learn the argument syntax	GPT-4o, given a bundled copy of the manual (v1's `doc_sources/man/convert.txt`, ImageMagick 6) and worked examples for `touch`, `rm`, `mv` and `ls`, writes the grammar; it is imported and checked, with up to three tries (section 3). It is told to use only what the manual shows. v1's model-written grammar and v1's curated reference both list the manual's 248 names, with `-connected-component` and `-view` as the manual has them, no `+` forms, and one operand node	Starts from the same kind of manual, then checks it against `-help`, the library's strings and about 3,700 runs: 261 names, both forms, word counts, and value lists from the binary
2. Choose what to run	Code enumerates combinations of up to N flags with typed values, paired with input files from a fixed set that includes a JPEG (section 4). For `convert`, `caruca generate` gives 7,308 command lines at one flag and 917,154 at two (measured on the Mac)	One option at a time, with 0 to 3 extra words; a few behaviour checks pair two options; never combinations
3. Run and observe	Each case in its own OverlayFS layer, in a fresh namespace, under `chroot`, with `strace` recording every file read and write (section 5). The paper projects 98.72 hours for `convert` at two flags, its only command over a day, and adds that the combinations "exaggerate its complexity without revealing meaningful behavioral variation" (section 7.4)	A plain `/tmp/cvt` folder; error messages and whether `out.png` appeared; no tracing; 21.5 minutes of session time
4. Decide the behaviour	Rules over the traces (section 6.3). v1's first committed annotation for `convert` (2024-11-28) reads "Found no successful runs. Defaulting to side-effectful"	Reasoned: every operand but the last is read, the last is written; standard streams for `-`, `fmt:-`, `-identify`, `-print`, `-write png:-`; not splittable

JOB	V1, AS THE PAPER DESIGNS IT	RUN 8'S AGENT
5. Write for consumers	Adapters to PaSh, Shseer, POSH and ShellCheck (section 6.4). v1's committed PaSh annotation for `convert` (`caruca/save/convert.json`) has had no cases since 2024-12-04, and `benchmarks/MILESTONES` lists `convert` as not done. An older hand-written PaSh annotation in v1's `eval/pash-annotations/convert.json` says `pure`, reading the first argument and writing the second	PaSh's current Python form: a 426-entry table and two classes, loadable in PaSh 0.2.4

Where the two designs spend their effort. The paper's cost for `convert` comes from combining options: two at a time already means 917,154 command lines. It notes that this cost buys little, because `convert`'s file behaviour is the same whatever the options. The agent spent its effort on the other axis: one option at a time, to learn which words are options and how many words each consumes. That is the part v1 hands to its model and does not check against the binary. Neither design, as run, checked the file behaviour of real `convert` invocations: v1's runs did not succeed, and the agent reasoned it.

Where the formats fall short for this command. `convert` puts options between files and gives some options two words. v1's syntax format can place options between files, with more positions, and v1's authors did so in the grammar they wrote for PaSh's tests (input, `-resize`, output); it cannot express two-word options. v1's curated reference does not use positions this way: one operand node of one or more paths, with no output marked. The agent marks the output, as a second operand node, but keeps all options before all files, as the samples do. PaSh's parser table can express neither, and in PaSh's real parser this decides the result: the usual order turns option words into file names. v1's PaSh tracing script (`caruca/run.sh`, `caruca trace --pash`) uses the three-position grammar with its full content pool (`--content split`, which includes the one JPEG); why v1 recorded "no successful runs" was not established.

What is "correct" depends on the yardstick. Scored against v1's reference, the agent loses one name and adds 275 spellings v1 does not have, and loses `-view`. Scored against the binary, the one missing name does not exist, the added names all run, and `-view` does take a value. This is the same tension as in run 5, larger.

Cost. The paper reports CPU time for its pipeline and not the cost of its model step. Run 8 used 21.5 minutes of session time and $6.04 at list price.

Run 8 compared with runs 4 and 5

Same model, effort, prompt, image and day.

MEASURE	RUN 4 (GREP)	RUN 5 (RM)	RUN 8 (CONVERT)
Session time · turns · tool calls	322 s · 11 · 10	60 s · 6 · 5	1,291 s · 34 · 33
Output tokens (thinking) · cost	24,495 (6,663) · $2.54	4,623 (937) · $0.80	46,065 (20,780) · $6.04
A sample of the same command	yes	yes	no
Option spellings (the command's interface)	84	17	522
Binary runs before writing	72	27	~3,700
Isolation	files in `/tmp`	one `mktemp -d` folder	a fixed `/tmp/cvt` folder
Test after writing	one harness run, against stand-ins	none	three harness runs, ~3,100 binary runs, against stand-ins
Version named in output files	3 of 4	1 of 4	1 of 4
Report overstates	"Everything passed" (one failure)	a bare `--interactive` means "always" (manual, not tested)	"All 522 spellings passed" (509 run, one flagged)

With a sample in front of it (runs 4 and 5), the agent kept the sample's table unchanged and changed only what a probe contradicted. Without one, it built everything from the binary, at roughly 2.4 to 7.5 times the cost of those runs.

What this run suggests we ask next

1. **Score it**, deciding the yardstick first: v1's reference for `convert` contains a name the binary rejects and a flag that takes a value.
2. **Decide what PaSh should receive for commands with options after files.** The format document does not say that PaSh's parser stops at the first file; a session cannot learn it without PaSh's code. Nor does it say a syntax may have more than two positions in practice; every sample uses two.
3. **Run `convert` with prompt version 2**, which asks for execution as the standard of evidence and for reproducibility, to see whether it traces file behaviour or isolates its probes.
4. **Keep the Mac awake for runs**: on mains power, or with a power assertion that holds on battery. The cost guard used here (a script on the Mac that read the run's event stream every two minutes and estimated spend from its token counts) worked and could join the launcher.
5. **Watch the "run every word" pattern** on a command whose options can act on the system.

Diagram tooling

The three diagrams were drawn with the Excalidraw diagram skill (`https://github.com/coleam00/excalidraw-diagram-skill`, at `.claude/skills/excalidraw-diagram/`). **The skill's repository declares no licence**; it is listed as third-party, unlicensed material in `LICENSE-TEMPLATES.md`, and is kept in the repository by Tiran's decision of 2026-10-13. Elements were placed by hand in a compact spec (`<name>.spec.json`, beside each diagram) and expanded with `naive_llm_caruca/excalidraw_fill.py`; each was rendered with `uv run python render_excalidraw.py <file>`, inspected, and corrected once or twice.